Try the new high-speed, cost-effective Veo 3.1 Lite (/web/20260413040728/https://ai.google.dev/gemini-api/docs/models/veo-3.1-lite-generate-preview) model for video generation at scale.

Gemini thinking

The Gemini 3 and 2.5 series models (/web/20260413040728/https://ai.google.dev/gemini-api/docs/models) use an internal "thinking process" that significantly improves their reasoning and multi-step planning abilities, making them highly effective for complex tasks such as coding, advanced mathematics, and data analysis.

This guide shows you how to work with Gemini's thinking capabilities using the Gemini API.

Generating content with thinking

Initiating a request with a thinking model is similar to any other content generation request. The key difference lies in specifying one of the models with thinking support (#supported-models) in the `model` field, as demonstrated in the following text generation (/web/20260413040728/https://ai.google.dev/gemini-api/docs/text-generation#text-input) example:

Python JavaScript... Go... REST... (https://web.archive.org/web/20260413040728/https://ai.google.dev/gemini-api/docs/thinking#python)

```
from google import genai

client = genai.Client()
prompt = "Explain the concept of Occam's Razor and provide a simple, everyday example."
```

```
response = client.models.generate_content(
    model="gemini-3-flash-preview",
    contents=prompt
)

print(response.text)
```

Thought summaries

Thought summaries are summarized versions of the model's raw thoughts and offer insights into the model's internal reasoning process. Note that thinking levels and budgets apply to the model's raw thoughts and not to thought summaries.

You can enable thought summaries by setting `includeThoughts` to `true` in your request configuration. You can then access the summary by iterating through the `response` parameter's `parts`, and checking the `thought` boolean.

Here's an example demonstrating how to enable and retrieve thought summaries without streaming, which returns a single, final thought summary with the response:

Python JavaScript... Go...
(https://web.archive.org/web/20260413040728/https://ai.google.dev/gemini-api/docs/thinking#python)

```
from google import genai
from google.genai import types

client = genai.Client()
prompt = "What is the sum of the first 50 prime numbers?"
response = client.models.generate_content(
```

```
  model="gemini-3-flash-preview",
  contents=prompt,
  config=types.GenerateContentConfig(
    thinking_config=types.ThinkingConfig(
      include_thoughts=True
    )
  )
)

for part in response.candidates[0].content.parts:
  if not part.text:
    continue
  if part.thought:
    print("Thought summary:")
    print(part.text)
    print()
  else:
    print("Answer:")
    print(part.text)
    print()
```

And here is an example using thinking with streaming, which returns rolling, incremental summaries during generation:

Python JavaScript... Go...
(https://web.archive.org/web/20260413040728/https://ai.google.dev/gemini-api/docs/thinking#python)

```
from google import genai
from google.genai import types

client = genai.Client()
```

```
prompt = """
Alice, Bob, and Carol each live in a different house on the same street: red, green, and blue.
The person who lives in the red house owns a cat.
Bob does not live in the green house.
Carol owns a dog.
The green house is to the left of the red house.
Alice does not own a cat.
Who lives in each house, and what pet do they own?
"""

thoughts = ""
answer = ""

for chunk in client.models.generate_content_stream(
    model="gemini-3-flash-preview",
    contents=prompt,
    config=types.GenerateContentConfig(
      thinking_config=types.ThinkingConfig(
        include_thoughts=True
      )
    )
):
  for part in chunk.candidates[0].content.parts:
    if not part.text:
      continue
    elif part.thought:
      if not thoughts:
        print("Thoughts summary:")
      print(part.text)
      thoughts += part.text
    else:
```

```
        if not answer:
          print("Answer:")
        print(part.text)
        answer += part.text
```

Controlling thinking

Gemini models engage in dynamic thinking by default, automatically adjusting the amount of reasoning effort based on the complexity of the user's request. However, if you have specific latency constraints or require the model to engage in deeper reasoning than usual, you can optionally use parameters to control thinking behavior.

Thinking levels (Gemini 3)

The **`thinkingLevel`** parameter, recommended for Gemini 3 models and onwards, lets you control reasoning behavior.

The following table details the **`thinkingLevel`** settings for each model type:

Thinking Level	Gemini 3.1 Pro	Gemini 3.1 Flash-Lite	Gemini 3 Flash	Description
`minimal`	Not supported	Supported (Default)	Supported	Matches the "no thinking" setting for most queries. The model may think very minimally for complex coding tasks. Minimizes latency for chat or high throughput applications. Note, **`minimal`** does not guarantee that thinking is off.

Thinking Level	Gemini 3.1 Pro	Gemini 3.1 Flash-Lite	Gemini 3 Flash	Description
`low`	Supported	Supported	Supported	Minimizes latency and cost. Best for simple instruction following, chat, or high-throughput applications.
`medium`	Supported	Supported	Supported	Balanced thinking for most tasks.
`high`	Supported (Default, Dynamic)	Supported (Dynamic)	Supported (Default, Dynamic)	Maximizes reasoning depth. The model may take significantly longer to reach a first (non thinking) output token, but the output will be more carefully reasoned.

The following example shows how to set the thinking level.

Python JavaScript... Go... REST...
(https://web.archive.org/web/20260413040728/https://ai.google.dev/gemini-api/docs/thinking#python)

```
from google import genai
from google.genai import types

client = genai.Client()

response = client.models.generate_content(
    model="gemini-3-flash-preview",
    contents="Provide a list of 3 famous physicists and their key contributions",
    config=types.GenerateContentConfig(
```

```
        thinking_config=types.ThinkingConfig(thinking_level="low")
    ),
)

print(response.text)
```

You cannot disable thinking for Gemini 3.1 Pro. Gemini 3 Flash and Flash-Lite also do not support full thinking-off, but the `minimal` setting means the model likely will not think (though it still potentially can). If you don't specify a thinking level, Gemini will use the Gemini 3 models' default dynamic thinking level, "`high`".

Gemini 2.5 series models don't support `thinkingLevel`; use `thinkingBudget` instead.

Thinking budgets

The `thinkingBudget` parameter, introduced with the Gemini 2.5 series, guides the model on the specific number of thinking tokens to use for reasoning.

Note: Use the `thinkingLevel` parameter with Gemini 3 models. While `thinkingBudget` is accepted for backwards compatibility, using it with Gemini 3 Pro may result in unexpected performance.

The following are `thinkingBudget` configuration details for each model type. You can disable thinking by setting `thinkingBudget` to 0. Setting the `thinkingBudget` to -1 turns on **dynamic thinking**, meaning the model will adjust the budget based on the complexity of the request.

Model	Default setting (Thinking budget is not set)	Range	Disable thinking	Turn on dynamic thinking
2.5 Pro	Dynamic thinking	128 to 32768	N/A: Cannot disable thinking	`thinkingBudget = -1` (Default)
2.5 Flash	Dynamic thinking	0 to 24576	`thinkingBudget = 0`	`thinkingBudget = -1` (Default)
2.5 Flash Preview	Dynamic thinking	0 to 24576	`thinkingBudget = 0`	`thinkingBudget = -1` (Default)
2.5 Flash Lite	Model does not think	512 to 24576	`thinkingBudget = 0`	`thinkingBudget = -1`
2.5 Flash Lite Preview	Model does not think	512 to 24576	`thinkingBudget = 0`	`thinkingBudget = -1`
Robotics-ER 1.5 Preview	Dynamic thinking	0 to 24576	`thinkingBudget = 0`	`thinkingBudget = -1` (Default)
2.5 Flash Live Native Audio Preview (09-2025)	Dynamic thinking	0 to 24576	`thinkingBudget = 0`	`thinkingBudget = -1` (Default)

Python JavaScript... Go... REST...
(https://web.archive.org/web/20260413040728/https://ai.google.dev/gemini-api/docs/thinking#python)

```
from google import genai
from google.genai import types

client = genai.Client()
```

```
response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents="Provide a list of 3 famous physicists and their key contributions",
    config=types.GenerateContentConfig(
        thinking_config=types.ThinkingConfig(thinking_budget=1024)
        # Turn off thinking:
        # thinking_config=types.ThinkingConfig(thinking_budget=0)
        # Turn on dynamic thinking:
        # thinking_config=types.ThinkingConfig(thinking_budget=-1)
    ),
)

print(response.text)
```

Depending on the prompt, the model might overflow or underflow the token budget.

Thought signatures

Important: The Google GenAI SDK (/web/20260413040728/https://ai.google.dev/gemini-api/docs/libraries) automatically handles the return of thought signatures for you. You only need to manage thought signatures manually (/web/20260413040728/https://ai.google.dev/gemini-api/docs/function-calling#thought-signatures) if you're modifying conversation history or using the REST API.

The Gemini API is stateless, so the model treats every API request independently and doesn't have access to thought context from previous turns in multi-turn interactions.

In order to enable maintaining thought context across multi-turn interactions, Gemini returns thought signatures, which are encrypted representations of the model's internal thought process.

- **Gemini 2.5 models** return thought signatures when thinking is enabled and the request includes function calling (/web/20260413040728/https://ai.google.dev/gemini-api/docs/function-calling#thinking), specifically function declarations (/web/20260413040728/https://ai.google.dev/gemini-api/docs/function-calling#step-2).
- **Gemini 3 models** may return thought signatures for all types of parts (/web/20260413040728/https://ai.google.dev/api/caching#Part). We recommend you always pass all signatures back as received, but it's *required* for function calling signatures. Read the Thought Signatures (/web/20260413040728/https://ai.google.dev/gemini-api/docs/thought-signatures) page to learn more.

Other usage limitations to consider with function calling include:

- Signatures are returned from the model within other parts in the response, for example function calling or text parts. Return the entire response (https://web.archive.org/web/20260413040728/https://ai.google.dev/gemini-api/docs/function-calling#step-4) with all parts back to the model in subsequent turns.
- Don't concatenate parts with signatures together.
- Don't merge one part with a signature with another part without a signature.

Pricing

Note: Summaries are available in the free and paid tiers (/web/20260413040728/https://ai.google.dev/gemini-api/docs/pricing) of the API. **Thought signatures** will increase the input tokens you are charged when sent back as part of the request.

When thinking is turned on, response pricing is the sum of output tokens and thinking tokens. You can get the total number of generated thinking tokens from the `thoughtsTokenCount` field.

Python JavaScript... Go...
(https://web.archive.org/web/20260413040728/https://ai.google.dev/gemini-api/docs/thinking#python)

```
# ...
print("Thoughts tokens:",response.usage_metadata.thoughts_token_count)
print("Output tokens:",response.usage_metadata.candidates_token_count)
```

Thinking models generate full thoughts to improve the quality of the final response, and then output summaries (#summaries) to provide insight into the thought process. So, pricing is based on the full thought tokens the model needs to generate to create a summary, despite only the summary being output from the API.

You can learn more about tokens in the Token counting (/web/20260413040728/https://ai.google.dev/gemini-api/docs/tokens) guide.

Best practices

This section includes some guidance for using thinking models efficiently. As always, following our prompting guidance and best practices (/web/20260413040728/https://ai.google.dev/gemini-api/docs/prompting-strategies) will get you the best results.

Debugging and steering

- **Review reasoning**: When you're not getting your expected response from the thinking models, it can help to carefully analyze Gemini's thought summaries. You can see how it broke down the task and arrived at its conclusion, and use that information to correct

towards the right results.

- **Provide Guidance in Reasoning**: If you're hoping for a particularly lengthy output, you may want to provide guidance in your prompt to constrain the amount of thinking (#set-budget) the model uses. This lets you reserve more of the token output for your response.

Task complexity

- **Easy Tasks (Thinking could be OFF):** For straightforward requests where complex reasoning isn't required, such as fact retrieval or classification, thinking is not required. Examples include:
  - "Where was DeepMind founded?"
  - "Is this email asking for a meeting or just providing information?"
- **Medium Tasks (Default/Some Thinking):** Many common requests benefit from a degree of step-by-step processing or deeper understanding. Gemini can flexibly use thinking capability for tasks like:
  - Analogize photosynthesis and growing up.
  - Compare and contrast electric cars and hybrid cars.
- **Hard Tasks (Maximum Thinking Capability):** For truly complex challenges, such as solving complex math problems or coding tasks, we recommend setting a high thinking budget. These types of tasks require the model to engage its full reasoning and planning capabilities, often involving many internal steps before providing an answer. Examples include:
  - Solve problem 1 in AIME 2025: Find the sum of all integer bases b > 9 for which 17_b is a divisor of 97_b.
  - Write Python code for a web application that visualizes real-time stock market data, including user authentication. Make it as efficient as possible.

Supported models, tools, and capabilities

Thinking features are supported on all 3 and 2.5 series models. You can find all model capabilities on the model overview (/web/20260413040728/https://ai.google.dev/gemini-api/docs/models) page.

Thinking models work with all of Gemini's tools and capabilities. This allows the models to interact with external systems, execute code, or access real-time information, incorporating the results into their reasoning and final response.

You can try examples of using tools with thinking models in the Thinking cookbook (https://web.archive.org/web/20260413040728/https://colab.sandbox.google.com/github/google-gemini/cookbook/blob/main/quickstarts/Get_started_thinking.ipynb)
.

What's next?

- Thinking coverage is available in our OpenAI Compatibility (/web/20260413040728/https://ai.google.dev/gemini-api/docs/openai#thinking) guide.

Last updated 2026-03-25 UTC.